

ALLIANCE UNIVERSITY

Private University established in Karnataka State by Act No.34 of year 2010
Recognized by the University Grants Commission (UGC), New Delhi

Database Management Systems

AIRLINE RESERVATION SYSTEM

A Relational Database Design for Flight Booking & Management

Bachelor of Computer Application
Semester – III

R MONISHA BANGERA
2511021240053

Submitted To -

Faculty Name: Renita Joy R

Faculty Signature:

Basic SQL Queries and Aggregate Functions

1. Introduction

This report shows SQL queries run on the Airline Reservation System database (`airline_reservation_system`, loaded with `sample_data_insertion.sql`: 20 passengers, 28 flights, 50 bookings and so on). The queries demonstrate filtering with WHERE, sorting with ORDER BY, grouping with GROUP BY, filtering groups with HAVING, and the aggregate functions COUNT(), SUM(), AVG(), MIN() and MAX(). Each query is followed by its output and a short explanation. Section 7 adds INSERT, UPDATE and DELETE statements. All commands are also in the script `basic_queries.sql`.

Order in which SQL processes a query: FROM and JOIN, then WHERE (filters rows), GROUP BY (forms groups), aggregate functions, HAVING (filters groups), SELECT, ORDER BY (sorts), LIMIT. This explains why WHERE cannot use COUNT() or AVG(), while HAVING can.

2. WHERE

Q1. Airports in India with three or more terminals

```
SELECT Airport_Code, Airport_Name, City, Terminal_Count
FROM Airport
WHERE Country = 'India' AND Terminal_Count >= 3;
```

Airport_Code	Airport_Name	City	Terminal_Coun
DEL	Indira Gandhi International Airport	Delhi	t 3
MAA	Chennai International Airport	Chennai	4
COK	Cochin International Airport	Kochi	3

Explanation: WHERE keeps only rows that satisfy both conditions joined by AND.

Q2. Flights that are not running on schedule

```
SELECT Flight_ID, Flight_Number, Departure_DateTime, Flight_Status
FROM Flight
WHERE Flight_Status IN ('Delayed', 'Cancelled');
```

Flight_ID	Flight_Number	Departure_DateTime	Flight_Status
113	UK821	2026-11-02 07:15:00	Delayed
121	AI935	2026-11-01 07:00:00	Delayed
126	6E640	2026-11-03 11:00:00	Cancelled

Explanation: IN tests a column against a list of values, a shorter form of several OR conditions.

Q3. Confirmed bookings with a fare between 10,000 and 25,000

```
SELECT Booking_ID, Passenger_ID, Flight_ID, Fare
FROM Booking
WHERE Booking_Status = 'Confirmed'
    AND Fare BETWEEN 10000 AND 25000;
```

Booking_ID	Passenger_ID	Flight_ID	Fare
5003	3	101	13800.0
5009	9	103	0
5017	14	107	14500.0
5023	18	111	0
5024	19	111	11900.0
5027	18	112	0
5032	8	114	21500.0
5038	14	120	0
5039	15	120	12800.0
			0
			21800.0
			0

Booking_ID	Passenger_ID	Flight_ID	Fare
5040	16	120	14800.00
5041	14	121	22000.00
5042	15	121	15200.00
5048	2	128	17800.00

Explanation: BETWEEN is inclusive of both ends. These are the Business and First class bookings.

Q4. Passengers whose last name starts with S

```
SELECT Passenger_ID, First_Name, Last_Name, Email
FROM Passenger
WHERE Last_Name LIKE 'S%';
```

Passenger_ID	First_Name	Last_Name	Email
2	Rahul	Sharma	rahul.sharma@example.com
6	Vikram	Singh	vikram.singh@example.com
7	Neha	Singh	neha.singh@example.com
10	Karthik	Subramania	karthik.subramanian@example.com
19	Fatima	n Sheikh	fatima.sheikh@example.com

Explanation: LIKE with % is a pattern match: S followed by any characters.

Q5. Bookings made on or after 20 October 2026

```
SELECT Booking_ID, Passenger_ID, Booking_Date, Booking_Status, Fare
FROM Booking
WHERE Booking_Date >= '2026-10-20'
    AND Booking_Status <> 'Cancelled';
```

Booking_ID	Passenger_ID	Booking_Date	Booking_Status	Fare
5006	6	2026-10-20	Confirmed	6100.00
5007	7	2026-10-20	Confirmed	6100.00
5008	8	2026-10-21	Pending	5900.00
5009	9	2026-10-22	Confirmed	14500.0
5010	10	2026-10-22	Confirmed	0
5021	16	2026-10-20	Confirmed	6400.00
5022	17	2026-10-20	Pending	3200.00
5037	11	2026-10-21	Pending	3200.00
5049	3	2026-10-20	Confirmed	3300.00
5050	4	2026-10-20	Confirmed	4300.00

4300.00

Explanation: Dates are compared like values; <> means not equal.

3. ORDER BY

Q6. Five most expensive bookings

```
SELECT Booking_ID, Flight_ID, Booking_Status, Fare
FROM Booking
ORDER BY Fare DESC, Booking_ID
LIMIT 5;
```

Booking_ID	Flight_ID	Booking_Status	Fare
5047	128	Confirmed	38500.00
5041	121	Confirmed	22000.00
5027	112	Confirmed	21800.00
5023	111	Confirmed	21500.00
5038	120	Confirmed	21500.00

Explanation: ORDER BY … DESC sorts from highest to lowest; the second sort key Booking_ID breaks ties; LIMIT 5 keeps the first five rows.

Q7. Passengers listed by last name, then first name

```
SELECT Passenger_ID, Last_Name, First_Name, Date_of_Birth
FROM Passenger
ORDER BY Last_Name ASC, First_Name ASC
LIMIT 8;
```

Passenger_ID	Last_Name	First_Name	Date_of_Birth
16	Babu	Suresh	1975-09-09
1	Bangera	Monisha	2005-03-14
8	Faizan	Mohammed	1995-12-25
15	Gupta	Ishita	1996-01-30
5	Iyer	Ananya	2001-09-17
9	Kulkarni	Sneha	1998-06-19
14	Mehta	Rohan	1982-07-07
11	Menon	Deepa	1979-10-05

Explanation: Sorting on several columns: when two passengers share a last name, the first name decides the order. LIMIT 8 shows only the first eight rows.

Q8. Bengaluru departures on 1 November 2026 in time order

```
SELECT Flight_ID, Flight_Number, Departure_DateTime, Arrival_DateTime,
Flight_Status
FROM Flight
WHERE Departure_Airport_ID = 1
   AND DATE(Departure_DateTime) = '2026-11-01'
ORDER BY Departure_DateTime;
```

Flight_ID	Flight_Number	Departure_DateTime	Arrival_DateTime	Flight_Status
120	AI934	2026-11-01 02:30:00	2026-11-01 05:10:00	Scheduled
101	AI502	2026-11-01 06:00:00	2026-11-01 08:50:00	Scheduled
117	IX771	2026-11-01 07:30:00	2026-11-01 08:40:00	Scheduled
107	6E204	2026-11-01 09:30:00	2026-11-01 11:20:00	Scheduled
124	6E412	2026-11-01 15:00:00	2026-11-01 16:05:00	Scheduled
110	6E310	2026-11-01 16:00:00	2026-11-01 17:20:00	Scheduled

Explanation: WHERE and ORDER BY together: filter first, then sort the surviving rows. Airport_ID 1 is BLR.

4. Aggregate functions

Q9. COUNT: size of the bookings data

```
SELECT COUNT(*)                                AS Total_Bookings,
       COUNT(DISTINCT Passenger_ID) AS Passengers_Who_Booked,
       COUNT(DISTINCT Flight_ID)          AS Flights_Booked
FROM Booking;
```

Total_Bookings	Passengers_Who_Booked	Flights_Booked
50	20	20

Explanation: COUNT(*) counts rows; COUNT(DISTINCT column) counts unique values only.

Q10. COUNT, SUM, AVG, MIN and MAX on confirmed fares

```
SELECT COUNT(*)                  AS Confirmed_Bookings,
       SUM(Fare)                 AS Total_Fare,
       ROUND(AVG(Fare), 2) AS Average_Fare,
       MIN(Fare)                 AS    Lowest_Fare,
       MAX(Fare)                 AS Highest_Fare
FROM Booking
WHERE Booking_Status = 'Confirmed';
```

Confirmed_Bookings	Total_Fare	Average_Fare	Lowest_Fare	Highest_Fare
43	400600.00	9316.28	2900.00	38500.00

Explanation: All five aggregate functions in one query. Each collapses many rows into one value; WHERE is applied first so cancelled and pending bookings are excluded.

Q11. Baggage summary

```
SELECT SUM(Baggage_Count)          AS Total_Bags,
       ROUND(AVG(Total_Weight),2)  AS Avg_Weight_Per_Booking_Kg,
       MAX(Total_Weight)           AS Heaviest_Booking_Kg,
       SUM(Excess_Baggage_Fee)     AS Excess_Fees_Collected
FROM Baggage;
```

Total_Bags	Avg_Weight_Per_Booking_Kg	Heaviest_Booking_Kg	Excess_Fees_Collected
60	22.15	46.50	12850.00

Explanation: Aggregates also work on other tables. AVG ignores NULLs but counts zero-weight rows.

Q12. Seating capacity of the active fleet

```
SELECT COUNT(*)             AS Active_Aircraft,
       SUM(Total_Seats) AS Total_Capacity,
       MIN(Total_Seats) AS Smallest_Aircraft,
       MAX(Total_Seats) AS Largest_Aircraft
FROM Aircraft
WHERE Aircraft_Status = 'Active';
```

Active_Aircraft	Total_Capacity	Smallest_Aircraft	Largest_Aircraft
7	1206	72	222

Explanation: The aircraft in Maintenance is excluded by the WHERE clause, so it does not add to the capacity.

5. GROUP BY

Q13. Bookings and fare total for each booking status

```
SELECT Booking_Status,
       COUNT(*) AS Bookings,
       SUM(Fare) AS Total_Fare
FROM Booking
GROUP  BY  Booking_Status
ORDER BY Bookings DESC;
```

Booking_Status	Bookings	Total_Fare
Confirmed	43	400600.00
Cancelled	4	17900.00
Pending	3	12400.00

Explanation: GROUP BY makes one output row per status; the aggregates are calculated separately inside each group.

Q14. Successful payments by payment method

```
SELECT Payment_Method,
       COUNT(*)                  AS Payments,
       SUM(Amount)               AS Total_Collected,
       ROUND(AVG(Amount), 2) AS Average_Payment
FROM Payment
WHERE Payment_Status = 'Successful'
GROUP BY Payment_Method
ORDER BY Total_Collected DESC;
```

Payment_Method	Payment	Total_Collected	Average_Payment
Card	s 16	179600.00	11225.00
Net Banking	16	161550.00	10096.88
UPI	15	90200.00	6013.33

Explanation: WHERE removes the failed and pending payments before the rows are grouped.

Q15. Confirmed fares by seat class (join with Seat)

```
SELECT s.Seat_Class,
       COUNT(*)                  AS Bookings,
       ROUND(AVG(b.Fare), 2) AS Average_Fare,
       MIN(b.Fare)               AS  Lowest_Fare,
       MAX(b.Fare)               AS Highest_Fare
FROM Booking b
JOIN  Seat  s  ON  s.Seat_ID  =  b.Seat_ID
WHERE  b.Booking_Status  =  'Confirmed'
GROUP BY s.Seat_Class
ORDER BY Average_Fare DESC;
```

Seat_Class	Bookings	Average_Fare	Lowest_Fare	Highest_Fare
First	2	21650.00	21500.00	21800.00
Business	8	18425.00	11900.00	38500.00
Economy	33	6360.61	2900.00	17800.00

Explanation: The class lives in Seat and the fare in Booking, so the tables are joined first and the joined rows are then grouped.

Q16. Number of flights assigned to each aircraft

```
SELECT a.Aircraft_ID, a.Aircraft_Model, a.Registration_No,
       a.Aircraft_Status, COUNT(f.Flight_ID) AS Flights
FROM Aircraft a
LEFT JOIN Flight f ON f.Aircraft_ID = a.Aircraft_ID
GROUP BY a.Aircraft_ID, a.Aircraft_Model, a.Registration_No,
a.Aircraft_Status
ORDER BY Flights DESC, a.Aircraft_ID;
```

Aircraft_ID	Aircraft_Model	Registration_No	Aircraft_Status	Flights
1	Airbus A320neo	VT-IAA	Active	6
2	Airbus A320neo	VT-IAB	Active	5
3	Airbus A321	VT-IAC	Active	4
8	Boeing 737-800	VT-SGC	Active	4
4	Boeing 737-800	VT-SGA	Active	3
6	ATR 72-600	V T- ATA	Active	3
7	Airbus A320neo	VT-IAD	Active	3
5	Boeing 737-800	VT-SGB	Maintenance	0

Explanation: LEFT JOIN keeps aircraft that have no flights, so the grounded aircraft appears with a count of 0 (an inner join would hide it).

6. HAVING

Q17. Flights with four or more bookings

```
SELECT f.Flight_ID, f.Flight_Number, f.Departure_DateTime,
       COUNT(*) AS Bookings
FROM Booking b
JOIN Flight f ON f.Flight_ID = b.Flight_ID
GROUP BY f.Flight_ID, f.Flight_Number, f.Departure_DateTime
HAVING COUNT(*) >= 4
ORDER BY Bookings DESC, f.Flight_ID;
```

Flight_ID	Flight_Number	Departure_DateTime	Bookings
101	AI502	2026-11-01 06:00:00	5
107	6E204	2026-11-01 09:30:00	4
111	UK821	2026-11-01 07:15:00	4

Explanation: HAVING filters groups after they are formed. WHERE cannot do this because COUNT(*) does not exist until rows are grouped.

Q18. Passengers with two or more confirmed bookings

```
SELECT p.Passenger_ID,
       CONCAT(p.First_Name, ' ', p.Last_Name) AS Passenger,
       COUNT(*)       AS Confirmed_Bookings,
       SUM(b.Fare) AS Total_Spent
FROM Passenger p
JOIN Booking b ON b.Passenger_ID = p.Passenger_ID
WHERE b.Booking_Status = 'Confirmed'
GROUP BY p.Passenger_ID, p.First_Name, p.Last_Name
HAVING COUNT(*) >= 2
ORDER BY Total_Spent DESC
LIMIT 6;
```

Passenger_ID	Passenger	Confirmed_Bookings	Total_Spent
18	Aarav Patel	3	81800.00
14	Rohan Mehta	3	55400.00
15	Ishita Gupta	2	30000.00
2	Rahul Sharma	3	29100.00
3	Priya Nair	3	23500.00
6	Vikram Singh	3	18200.00

Explanation: WHERE picks confirmed bookings (row filter), then HAVING keeps only repeat travellers (group filter). These are the airline's frequent customers.

Q19. Routes whose average confirmed fare is above 10,000

```
SELECT CONCAT(dep.Airport_Code, ' -> ', arr.Airport_Code) AS Route,
       COUNT(*)                          AS Bookings,
       ROUND(AVG(b.Fare), 2) AS Average_Fare
FROM Booking b
JOIN Flight f          ON f.Flight_ID          = b.Flight_ID
JOIN Airport dep ON dep.Airport_ID = f.Departure_Airport_ID
JOIN Airport arr ON arr.Airport_ID = f.Arrival_Airport_ID
WHERE b.Booking_Status = 'Confirmed'
GROUP BY dep.Airport_Code, arr.Airport_Code
HAVING AVG(b.Fare) > 10000
ORDER BY Average_Fare DESC;
```

Route	Bookings	Average_Fare
BLR -> SIN	2	28150.00
DXB -> BLR	2	18600.00
BLR -> DXB	3	17033.33
DEL -> BOM	2	13550.00
BOM -> DEL	5	10140.00

Explanation: Uses WHERE, GROUP BY, HAVING and ORDER BY together. HAVING can filter on an aggregate such as AVG that WHERE cannot use.

Q20. Flights carrying more than 50 kg of checked baggage in total

```
SELECT f.Flight_Number, f.Departure_DateTime,
       SUM(g.Baggage_Count) AS Bags,
       SUM(g.Total_Weight) AS Total_Weight_Kg
FROM Baggage g
JOIN Booking b ON b.Booking_ID = g.Booking_ID
JOIN Flight f ON f.Flight_ID = b.Flight_ID
GROUP BY f.Flight_ID, f.Flight_Number, f.Departure_DateTime
HAVING SUM(g.Total_Weight) > 50
ORDER BY Total_Weight_Kg DESC, f.Flight_ID;
```

Flight_Number	Departure_DateTime	Bags	Total_Weight_Kg
UK821	2026-11-01 07:15:00	7	109.70
AI934	2026-11-01 02:30:00	5	98.00
AI502	2026-11-01 06:00:00	6	90.70
AI935	2026-11-01 07:00:00	4	80.00
AI346	2026-11-03 23:55:00	4	72.00
SG145	2026-11-01 08:00:00	4	62.30
UK822	2026-11-01 11:00:00	4	61.50
6E204	2026-11-01 09:30:00	4	60.50
AI503	2026-11-01 10:00:00	4	52.40

Explanation: Three tables joined, grouped by flight, and filtered on a SUM. Useful for load planning.

7. Data manipulation: INSERT, UPDATE and DELETE

These statements change data, so they are wrapped in a transaction and undone with ROLLBACK, which leaves the sample data unchanged for the next run.

```
START TRANSACTION;

-- M1 INSERT a new passenger
INSERT INTO Passenger (First_Name, Last_Name, Email, Phone, Date_of_Birth,
Gender)
VALUES ('Divya', 'Shetty', 'divya.shetty@example.com', '9000010021', '2002-
08-18', 'Female');
SELECT Passenger_ID, First_Name, Last_Name, Email, Phone FROM Passenger WHERE
Passenger_ID > 20;
```

Passenger_ID	First_Name	Last_Name	Email	Phone
21	Divya	Shetty	divya.shetty@example.com	9000010021

AUTO_INCREMENT supplied Passenger_ID 21 automatically.

```
-- M2 UPDATE one passenger's phone number
UPDATE Passenger SET Phone = '9000099999' WHERE Email =
'divya.shetty@example.com';
SELECT Passenger_ID, First_Name, Last_Name, Phone FROM Passenger WHERE
Passenger_ID > 20;
```

Passenger_ID	First_Name	Last_Name	Phone
21	Divya	Shetty	9000099999

```
-- M3 UPDATE a flight status (the delayed flight is back on time)
UPDATE Flight SET Flight_Status = 'Scheduled' WHERE Flight_ID = 113;
SELECT Flight_ID, Flight_Number, Flight_Status FROM Flight WHERE Flight_ID=
113;
```

Flight_ID	Flight_Number	Flight_Status
113	UK821	Scheduled

Before this statement the status was Delayed (see Q2).

```
-- M4 UPDATE many rows: 10% discount on all pending bookings
UPDATE Booking SET Fare = ROUND(Fare * 0.90, 2) WHERE Booking_Status =
'Pending';
SELECT Booking_ID, Booking_Status, Fare FROM Booking WHERE Booking_Status=
'Pending' ORDER BY Booking_ID;
```

Booking_ID	Booking_Status	Fare
5008	Pending	5310.00
5022	Pending	2880.00
5037	Pending	2970.00

The fares were 5900.00, 3200.00 and 3300.00 before the update. Without a WHERE clause every booking would have been changed.

```
-- M5 DELETE the new passenger (allowed: no booking refers to it)
DELETE FROM Passenger WHERE Passenger_ID = 21;
SELECT COUNT(*) AS Total_Passengers FROM Passenger;

ROLLBACK;
SELECT COUNT(*) AS Total_Passengers FROM Passenger;
```

Total_Passengers
20

Both counts are 20: after the DELETE the new passenger is gone, and after ROLLBACK the original data is restored (flight 113 is Delayed again and the pending fares are back to normal). Deleting a passenger who has bookings, for example `DELETE FROM Passenger WHERE Passenger_ID = 1;`, would be rejected by the foreign key with error 1451, as shown in the data integrity tests.

8. Conclusion

The queries show how SQL retrieves exactly the data the airline needs. WHERE selects rows, ORDER BY arranges them, aggregate functions summarize them, GROUP BY summarizes per category (status, payment method, seat class, aircraft), and HAVING keeps only the groups that meet a condition, such as flights with at least four bookings or routes with a high average fare. Combined with joins, these clauses turn the nine related tables into answers about revenue, load, customer behaviour and fleet use.

SQL Code

```sql
-- ========================================================================
-- AIRLINE RESERVATION SYSTEM : Basic SQL Queries and Aggregate Functions
-- Student : R Monisha Bangera (2511021240053)
-- Run AFTER sample_data_insertion.sql. Each query is independent.
-- ========================================================================
USE airline_reservation_system;


-- ###################### WHERE ######################

-- Q1. Airports in India with three or more terminals
-- WHERE keeps only rows that satisfy both conditions joined by AND.
SELECT Airport_Code, Airport_Name, City, Terminal_Count
FROM Airport
WHERE Country = 'India' AND Terminal_Count >= 3;

-- Q2. Flights that are not running on schedule
-- IN tests a column against a list of values, a shorter form of several OR conditions.
SELECT Flight_ID, Flight_Number, Departure_DateTime, Flight_Status
FROM Flight
WHERE Flight_Status IN ('Delayed', 'Cancelled');

-- Q3. Confirmed bookings with a fare between 10,000 and 25,000
-- BETWEEN is inclusive of both ends. These are the Business and First class bookings.
SELECT Booking_ID, Passenger_ID, Flight_ID, Fare
FROM Booking
WHERE Booking_Status = 'Confirmed'
   AND Fare BETWEEN 10000 AND 25000;

-- Q4. Passengers whose last name starts with S
-- LIKE with % is a pattern match: S followed by any characters. SELECT Passenger_ID, First_Name, Last_Name, Email
FROM Passenger
WHERE Last_Name LIKE 'S%';

-- Q5. Bookings made on or after 20 October 2026
-- Dates are compared like values; <> means not equal.
SELECT Booking_ID, Passenger_ID, Booking_Date, Booking_Status, Fare
FROM Booking
WHERE Booking_Date >= '2026-10-20'
   AND Booking_Status <> 'Cancelled';


-- ###################### ORDER BY ######################

-- Q6. Five most expensive bookings
-- ORDER BY ... DESC sorts from highest to lowest; the second sort key Booking_ID breaks ties; LIMIT 5 keeps the first five rows.
```

```sql
SELECT Booking_ID, Flight_ID, Booking_Status, Fare
FROM Booking
ORDER BY Fare DESC, Booking_ID
LIMIT 5;

-- Q7. Passengers listed by last name, then first name
-- Sorting on several columns: rows with the same last name (the two Singhs,
the two Vermas) are then ordered by first name.
SELECT Passenger_ID, Last_Name, First_Name, Date_of_Birth
FROM Passenger
ORDER BY Last_Name ASC, First_Name ASC
LIMIT 8;

-- Q8. Bengaluru departures on 1 November 2026 in time order
-- WHERE and ORDER BY together: filter first, then sort the surviving rows.
Airport_ID 1 is BLR.
SELECT Flight_ID, Flight_Number, Departure_DateTime, Arrival_DateTime,
Flight_Status
FROM Flight
WHERE Departure_Airport_ID = 1
  AND DATE(Departure_DateTime) = '2026-11-01'
ORDER BY Departure_DateTime;


-- ###################### AGGREGATE FUNCTIONS ######################

-- Q9. COUNT: size of the bookings data
-- COUNT(*) counts rows; COUNT(DISTINCT column) counts unique values only.
SELECT COUNT(*)                          AS Total_Bookings,
       COUNT(DISTINCT Passenger_ID) AS Passengers_Who_Booked,
       COUNT(DISTINCT Flight_ID)       AS Flights_Booked
FROM Booking;

-- Q10. COUNT, SUM, AVG, MIN and MAX on confirmed fares
-- All five aggregate functions in one query. Each collapses many rows into
one value; WHERE is applied first so cancelled and pending bookings are
excluded.
SELECT COUNT(*)                AS Confirmed_Bookings,
       SUM(Fare)               AS Total_Fare,
       ROUND(AVG(Fare), 2) AS Average_Fare,
       MIN(Fare)               AS  Lowest_Fare,
       MAX(Fare)               AS Highest_Fare
FROM Booking
WHERE Booking_Status = 'Confirmed';

-- Q11. Baggage summary
-- Aggregates also work on other tables. AVG ignores NULLs but counts zero-
weight rows.
SELECT SUM(Baggage_Count)              AS Total_Bags,
       ROUND(AVG(Total_Weight), 2)     AS Avg_Weight_Per_Booking_Kg,
       MAX(Total_Weight)               AS Heaviest_Booking_Kg,
       SUM(Excess_Baggage_Fee)         AS Excess_Fees_Collected
FROM Baggage;
```

```sql
-- Q12. Seating capacity of the active fleet
-- The aircraft in Maintenance is excluded by the WHERE clause, so it does not add to the capacity.
SELECT COUNT(*)             AS Active_Aircraft,
       SUM(Total_Seats) AS Total_Capacity,
       MIN(Total_Seats) AS Smallest_Aircraft,
       MAX(Total_Seats) AS Largest_Aircraft
FROM Aircraft
WHERE Aircraft_Status = 'Active';


-- ###################### GROUP BY ######################

-- Q13. Bookings and fare total for each booking status
-- GROUP BY makes one output row per status; the aggregates are calculated separately inside each group.
SELECT Booking_Status,
       COUNT(*)   AS Bookings,
       SUM(Fare) AS Total_Fare
FROM Booking
GROUP BY Booking_Status
ORDER BY Bookings DESC;

-- Q14. Successful payments by payment method
-- WHERE removes the failed and pending payments before the rows are grouped.
SELECT Payment_Method,
       COUNT(*)                AS Payments,
       SUM(Amount)             AS Total_Collected,
       ROUND(AVG(Amount), 2) AS Average_Payment
FROM Payment
WHERE Payment_Status = 'Successful'
GROUP BY Payment_Method
ORDER BY Total_Collected DESC;

-- Q15. Confirmed fares by seat class (join with Seat)
-- The class lives in Seat and the fare in Booking, so the tables are joined first and the joined rows are then grouped.
SELECT s.Seat_Class,
       COUNT(*)                AS Bookings,
       ROUND(AVG(b.Fare), 2) AS Average_Fare,
       MIN(b.Fare)             AS  Lowest_Fare,
       MAX(b.Fare)             AS Highest_Fare
FROM Booking b
JOIN Seat s ON s.Seat_ID = b.Seat_ID
WHERE b.Booking_Status = 'Confirmed'
GROUP BY s.Seat_Class
ORDER BY Average_Fare DESC;

-- Q16. Number of flights assigned to each aircraft
-- LEFT JOIN keeps aircraft that have no flights, so the grounded aircraft appears with a count of 0 (an inner join would hide it).
SELECT a.Aircraft_ID, a.Aircraft_Model, a.Registration_No,
       a.Aircraft_Status, COUNT(f.Flight_ID) AS Flights
FROM Aircraft a
```

```sql
LEFT JOIN Flight f ON f.Aircraft_ID = a.Aircraft_ID
GROUP BY a.Aircraft_ID, a.Aircraft_Model, a.Registration_No, a.Aircraft_Status ORDER BY Flights DESC, a.Aircraft_ID;


-- ###################### HAVING ######################

-- Q17. Flights with four or more bookings
-- HAVING filters groups after they are formed. WHERE cannot do this because COUNT(*) does not exist until rows are grouped.
SELECT f.Flight_ID, f.Flight_Number, f.Departure_DateTime,
       COUNT(*) AS Bookings
FROM Booking b
JOIN Flight f ON f.Flight_ID = b.Flight_ID
GROUP BY f.Flight_ID, f.Flight_Number, f.Departure_DateTime
HAVING COUNT(*) >= 4
ORDER BY Bookings DESC, f.Flight_ID;

-- Q18. Passengers with two or more confirmed bookings
-- WHERE picks confirmed bookings (row filter), then HAVING keeps only repeat travellers (group filter). These are the airline's frequent customers.
SELECT p.Passenger_ID,
       CONCAT(p.First_Name, ' ', p.Last_Name) AS Passenger,
       COUNT(*)      AS Confirmed_Bookings,
       SUM(b.Fare) AS Total_Spent
FROM Passenger p
JOIN Booking b ON b.Passenger_ID = p.Passenger_ID
WHERE b.Booking_Status = 'Confirmed'
GROUP BY p.Passenger_ID, p.First_Name, p.Last_Name
HAVING COUNT(*) >= 2
ORDER BY Total_Spent DESC
LIMIT 6;

-- Q19. Routes whose average confirmed fare is above 10,000
-- Uses WHERE, GROUP BY, HAVING and ORDER BY together. HAVING can filter on an aggregate such as AVG that WHERE cannot use.
SELECT CONCAT(dep.Airport_Code, ' -> ', arr.Airport_Code) AS Route,
       COUNT(*)                    AS Bookings,
       ROUND(AVG(b.Fare), 2) AS Average_Fare
FROM Booking b
JOIN Flight f         ON f.Flight_ID         = b.Flight_ID
JOIN Airport dep ON dep.Airport_ID = f.Departure_Airport_ID
JOIN Airport arr ON arr.Airport_ID = f.Arrival_Airport_ID
WHERE b.Booking_Status = 'Confirmed'
GROUP BY dep.Airport_Code, arr.Airport_Code
HAVING AVG(b.Fare) > 10000
ORDER BY Average_Fare DESC;

-- Q20. Flights carrying more than 50 kg of checked baggage in total
-- Three tables joined, grouped by flight, and filtered on a SUM. Useful for load planning.
SELECT f.Flight_Number, f.Departure_DateTime,
       SUM(g.Baggage_Count) AS Bags,
       SUM(g.Total_Weight)    AS Total_Weight_Kg
```

```
FROM Baggage g
JOIN Booking b ON b.Booking_ID = g.Booking_ID
JOIN Flight f ON f.Flight_ID = b.Flight_ID
GROUP BY f.Flight_ID, f.Flight_Number, f.Departure_DateTime
HAVING SUM(g.Total_Weight) > 50
ORDER BY Total_Weight_Kg DESC, f.Flight_ID;



-- ------------------------------------------------------------------
-- DATA MANIPULATION (INSERT / UPDATE / DELETE) - run inside a transaction
-- so the sample data is restored by the ROLLBACK at the end.
-- ------------------------------------------------------------------
START TRANSACTION;

-- M1 INSERT a new passenger, then check it
INSERT INTO Passenger (First_Name, Last_Name, Email, Phone, Date_of_Birth, Gender)
VALUES ('Divya', 'Shetty', 'divya.shetty@example.com', '9000010021', '2002-08- 18',
'Female');
SELECT Passenger_ID, First_Name, Last_Name, Email, Phone FROM Passenger WHERE
Passenger_ID > 20;

-- M2 UPDATE one passenger's phone number
UPDATE Passenger SET Phone = '9000099999' WHERE Email =
'divya.shetty@example.com';
SELECT Passenger_ID, First_Name, Last_Name, Phone FROM Passenger WHERE
Passenger_ID > 20;

-- M3 UPDATE a flight status (delayed flight is back on time)
SELECT Flight_ID, Flight_Number, Flight_Status FROM Flight WHERE Flight_ID = 113;
UPDATE Flight SET Flight_Status = 'Scheduled' WHERE Flight_ID = 113;
SELECT Flight_ID, Flight_Number, Flight_Status FROM Flight WHERE Flight_ID = 113;

-- M4 UPDATE many rows: 10% promotional discount on all pending bookings SELECT
Booking_ID, Booking_Status, Fare FROM Booking WHERE Booking_Status = 'Pending'
ORDER BY Booking_ID;
UPDATE Booking SET Fare = ROUND(Fare * 0.90, 2) WHERE Booking_Status = 'Pending';
SELECT Booking_ID, Booking_Status, Fare FROM Booking WHERE Booking_Status =
'Pending' ORDER BY Booking_ID;

-- M5 DELETE the new passenger (allowed because no booking refers to it) DELETE
FROM Passenger WHERE Passenger_ID = 21;
SELECT COUNT(*) AS Total_Passengers FROM Passenger;

ROLLBACK;
SELECT COUNT(*) AS Total_Passengers FROM Passenger;



                -- undo everything above
                                                    -- back to 20
```